

Bank Account Multithreading Project

📋 Project Overview

Create a multithreaded bank account simulation that demonstrates race conditions, synchronization, and thread-safe operations. This project will help you master the core concepts of concurrent programming in Java.

📋 Requirements

Phase 1: The Problem (Race Condition Demo)

Objective: Create a system that shows what happens WITHOUT proper synchronization.

1.1 BankAccount Class

Create a `BankAccount` class with:

- `accountNumber` (String)
- `balance` (double)
- `deposit(double amount)` method
- `withdraw(double amount)` method
- `getBalance()` method

1.2 Transaction Threads

Create multiple threads that perform transactions:

- **3-5 deposit threads** - Each deposits money repeatedly
- **3-5 withdrawal threads** - Each withdraws money repeatedly
- All threads operate on the **same account**

1.3 Expected Behavior (Without Sync)

- Initial balance: \$1000
 - Each thread performs 100 transactions
 - Final balance should be predictable, but **won't be** due to race conditions
 - Demonstrate the problem clearly
-

Phase 2: The Solution (Synchronized Version)

Objective: Fix the race condition using synchronization.

2.1 Synchronized Methods

Implement synchronized versions:

- `synchronized deposit()`
- `synchronized withdraw()`

2.2 Verification

- Run the same test as Phase 1
 - Final balance should now be **correct and consistent**
 - Compare results with Phase 1
-

Phase 3: Advanced Features

Objective: Add real-world banking features.

3.1 Multiple Accounts

- Create 3-5 different accounts
- Implement `transfer(BankAccount from, BankAccount to, double amount)`
- Avoid **deadlock** when transferring between accounts

3.2 Account Types

Create different account types:

- **SavingsAccount** - Has interest rate
- **CheckingAccount** - Has overdraft limit
- Use inheritance/polymorphism

3.3 Transaction History

- Log all transactions with timestamp
- Thread-safe logging mechanism
- Print transaction history at the end

Phase 4: Performance Comparison

Objective: Compare different synchronization approaches.

4.1 Implement Three Versions

1. **No synchronization** (race condition)
2. **Synchronized methods** (method-level locking)
3. **ReentrantLock** (explicit locking)

4.2 Measure Performance

For each version, measure:

- Execution time
- Final balance accuracy
- Number of successful transactions

4.3 Analysis

Write a brief analysis comparing:

- Correctness
- Performance
- Complexity

📄 Design Guidelines

Class Structure

```
BankAccountProject/
├── models/
│   ├── BankAccount.java
│   ├── SavingsAccount.java
│   └── CheckingAccount.java
├── threads/
│   ├── DepositThread.java
│   ├── WithdrawThread.java
│   └── TransferThread.java
├── sync/
│   ├── UnsynchronizedAccount.java
│   ├── SynchronizedAccount.java
│   └── ReentrantLockAccount.java
├── utils/
│   └── TransactionLogger.java
└── Main.java
```

📄 Expected Output

Phase 1 Output (No Sync)

```
=== Bank Account Simulation (NO SYNCHRONIZATION) ===
```

```
Initial Balance: $1000.00
```

```
Starting 5 deposit threads (each deposits $10, 100 times)...
```

```
Starting 5 withdraw threads (each withdraws $10, 100 times)...
```

```
[Thread-0] Deposited $10.00, Balance: $1010.00
```

```
[Thread-1] Withdrew $10.00, Balance: $1000.00
```

```
[Thread-2] Deposited $10.00, Balance: $1010.00
```

```
...
```

```
Expected Final Balance: $1000.00
```

```
Actual Final Balance: $987.50 ❌ INCORRECT!
```

```
Difference: -$12.50 (Race condition detected!)
```

Phase 2 Output (Synchronized)

```
=== Bank Account Simulation (SYNCHRONIZED) ===
```

```
Initial Balance: $1000.00
```

```
Starting 5 deposit threads (each deposits $10, 100 times)...
```

```
Starting 5 withdraw threads (each withdraws $10, 100 times)...
```

```
[Thread-0] Deposited $10.00, Balance: $1010.00
```

```
[Thread-1] Withdrew $10.00, Balance: $1000.00
```

```
...
```

```
Expected Final Balance: $1000.00
```

```
Actual Final Balance: $1000.00 ✅ CORRECT!
```

```
Difference: $0.00 (No race condition!)
```

📌 Learning Objectives

By completing this project, you will:

- 📌 Understand **race conditions** and why they occur
- 📌 Master **synchronized methods** and blocks
- 📌 Learn about **deadlock** and how to avoid it
- 📌 Practice with **ReentrantLock** and explicit locking
- 📌 Implement **thread-safe** data structures
- 📌 Measure and compare **performance** of different approaches
- 📌 Apply **OOP principles** in concurrent programming

📌 Bonus Challenges

Challenge 1: Deadlock Demonstration

Create a scenario that intentionally causes deadlock, then fix it using:

- Lock ordering
- `tryLock()` with timeout

Challenge 2: Thread Pool

Use `ExecutorService` instead of manually creating threads:

- Fixed thread pool
- Cached thread pool
- Compare performance

Challenge 3: Atomic Operations

Implement a version using `AtomicLong` for the balance:

- No explicit locks needed
- Compare with synchronized version

Challenge 4: Read-Write Locks

Use `ReentrantReadWriteLock` :

- Multiple threads can read balance simultaneously
- Only one thread can write at a time
- Measure performance improvement

📦 Deliverables

1. **Source Code** - Well-organized, commented code
2. **README.md** - How to run the project
3. **RESULTS.md** - Performance comparison and analysis
4. **Screenshots** - Output from each phase

📋 Testing Checklist

- ☐ Phase 1 shows race condition (incorrect final balance)
- ☐ Phase 2 fixes race condition (correct final balance)
- ☐ Phase 3 transfers work without deadlock
- ☐ Phase 4 performance comparison completed
- ☐ All threads complete successfully
- ☐ No exceptions thrown
- ☐ Transaction history is accurate
- ☐ Code is well-documented

📌 Tips

1. **Start Simple** - Get Phase 1 working first
2. **Test Frequently** - Run after each change
3. **Use Logging** - Print what each thread is doing
4. **Be Patient** - Multithreading bugs can be tricky
5. **Ask Questions** - Don't hesitate to ask for help!

📖 Concepts to Review

Before starting, make sure you understand:

- Thread creation and lifecycle
- Race conditions
- `synchronized` keyword
- Monitor locks
- Deadlock
- `wait()` and `notify()`
- `ReentrantLock`
- Thread-safe collections

🕒 Estimated Time

- **Phase 1:** 1-2 hours
- **Phase 2:** 1 hour
- **Phase 3:** 2-3 hours
- **Phase 4:** 1-2 hours
- **Total:** 5-8 hours

Good luck! This project will solidify your understanding of multithreading! 🍀